```
export { createBookKinChart } from "./antv";
package com.elexvx.bookkin.ai;
import java.util.List;
import java.security.Principal;
import jakarta.validation.Valid;
import jakarta.validation.constraints.Max;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Size;
import org.springframework.context.annotation.Profile;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
@RestController
@Profile("api")
@RequestMapping("/api/v1/ai")
@PreAuthorize("hasAnyRole('OWNER','ADMIN')")
public class AiController {
    private final AiMetadataService ai;
    private final AiSettingsService settings;
    public AiController(AiMetadataService ai, AiSettingsService settings) {
        this.ai = ai;
        this.settings = settings;
    }
    @GetMapping("/providers")
    AiProviderList providers() { return new AiProviderList(ai.providers()); }
    @GetMapping("/settings")
    AiSettingsService.AiSettingsView settings() { return settings.view(); }
    @PutMapping("/settings")
    AiSettingsService.AiSettingsView update(@Valid @RequestBody UpdateAiSettingsRequest input, Principal
principal) {
        return settings.update(new AiSettingsService.AiSettingsInput(input.enabled(), input.autoMatch(),
input.maxCandidates(),
                input.timeoutSeconds(), input.providers().stream().map(provider -> new AiSettingsService.
AiProviderInput(
                        provider.id(), provider.label(), provider.type(), provider.enabled(), provider.ba
seUrl(), provider.model(),
                        provider.apiKey(), provider.clearApiKey())).toList()), principal.getName());
    }
    public record UpdateAiSettingsRequest(
            @NotNull Boolean enabled,
            @NotNull Boolean autoMatch,
            @NotNull @Min(1) @Max(10) Integer maxCandidates,
            @NotNull @Min(5) @Max(120) Integer timeoutSeconds,
            @NotNull @Size(max = 20) List<@Valid ProviderRequest> providers) {}
    public record ProviderRequest(
            @NotBlank @Size(max = 80) String id,
            @NotBlank @Size(max = 120) String label,
            @NotBlank @Size(max = 32) String type,
            boolean enabled,
```

```
            @Size(max = 500) String baseUrl,
            @Size(max = 200) String model,
            @Size(max = 500) String apiKey,
            boolean clearApiKey) {}
    public record AiProviderList(List<AiMetadataService.ProviderInfo> items) {}
}
import * as THREE from 'three';
import type {LibraryMaterials} from './materials';
import {EAST_UPPER_ROOM as U} from './longRoomEastUpperRoom';
import {EAST_SERVICE_ACCESS as A} from '../longRoomLayout';
import {createEastServiceStair} from './longRoomEastServiceStair';
import {markCameraCollider} from '../../virtual-library-collision';
/** First-to-second-floor connection only. Other source storeys remain review-only. */
export function createEastServiceCore(materials:LibraryMaterials) {
 const root=new THREE.Group();root.name='East first to second floor service core';
 const stair=createEastServiceStair(materials,[0,U.floor]);root.add(stair);
 const C=U.core,finish=materials.parchment.clone();finish.roughness=0.94;
 const box=(name:string,w:number,h:number,d:number,x:number,y:number,z:number,m:THREE.Material=finish)=>{
  const mesh=new THREE.Mesh(new THREE.BoxGeometry(w,h,d),m);mesh.name=name;mesh.position.set(x,y,z);
  mesh.castShadow=mesh.receiveShadow=true;root.add(mesh);
  markCameraCollider(mesh,{id:`east-service-core-${name}-${x}-${z}`,shape:'box',center:{x:0,y:0,z:0},size
:{x:w,y:h,z:d}});
 };
 for(const x of [C.north,C.south])box('Lower service core side wall',0.12,U.floor,C.east-C.west,x,U.floor
/2,(C.east+C.west)/2);
 box('Lower service core east wall',C.south-C.north,U.floor,0.12,(C.south+C.north)/2,U.floor/2,C.east);
 box('Service stair entry threshold',A.width,0.16,1.5,A.x,-0.08,0.55,materials.floor);
 // The northern portion remains reserved for the lift; close access to its void.
 box('Service core lift reservation',1.38,U.ceiling,C.east-C.west,-8.35,U.ceiling/2,(C.east+C.west)/2);
 const route=stair.userData.walkingSamples as {x:number;y:number;z:number}[];
 // Start beyond the aisle-end pilaster's camera clearance after the cases
 // were deepened, while remaining on the hall side of the service doorway.
 root.userData.walkingSamples=[{x:A.x,y:0,z:-0.6},{x:A.x,y:0,z:1.1},...route,
  {x:A.x,y:U.floor,z:2.5},{x:A.x,y:U.floor,z:3.5},{x:A.x,y:U.floor,z:4.5}];
 return root;
}
import * as THREE from 'three';
import type {LibraryMaterials} from './materials';
import {markCameraCollider} from '../../virtual-library-collision';
/** Y1.002/003 and XS-4A/B: independent winder stair beside the lift.
 * Levels are source annotations relative to the Long Room floor; plan dimensions,
 * riser count, rail section and intermediate winding heights remain estimates.
 * The live core uses only the first-to-second-floor flight. Other levels remain review-only. */
export const EAST_SERVICE_STAIR={levels:[-4.58,0,5.49,8.89,12.42],
  startX:-6.4,turnX:-3.7,centerZ:1.8,radius:0.7,width:0.86,winders:8} as const;
export function createEastServiceStair(materials:LibraryMaterials,levels:readonly number[]=EAST_SERVICE_S
TAIR.levels) {
  const S=EAST_SERVICE_STAIR,root=new THREE.Group();root.name='East independent service stair';
  root.userData.fidelity='Y1.002/003 and XS-4A/B topology; source floor levels; estimated stair dimension
s';
  const surfaces=new THREE.Group();surfaces.name='Service stair walking surfaces';root.add(surfaces);
  const route:THREE.Vector3[]=[];
  const posts:THREE.Matrix4[]=[];
  const geometry=new THREE.CylinderGeometry(0.013,0.013,1,8);
```

```
  const add=(name:string,g:THREE.BufferGeometry,m:THREE.Material,parent:THREE.Group=root)=>{
    const mesh=new THREE.Mesh(g,m);mesh.name=name;mesh.castShadow=mesh.receiveShadow=true;parent.add(mesh
);return mesh;
  };
  const box=(name:string,w:number,h:number,d:number,x:number,y:number,z:number,parent=surfaces)=>{
    const m=add(name,new THREE.BoxGeometry(w,h,d),materials.woodWarm,parent);m.position.set(x,y,z);return
 m;
  };
  const rails=(points:THREE.Vector3[])=>{
    const curve=new THREE.CurvePath<THREE.Vector3>();
    for(let i=1;i<points.length;i++)curve.add(new THREE.LineCurve3(points[i-1],points[i]));
    add('Service stair continuous round handrail',new THREE.TubeGeometry(curve,points.length*3,0.03,8,fal
se),materials.woodWarm);
  };
  for(const y of levels)box('Service stair floor landing',1.05,0.16,2*S.radius+S.width,
    S.startX-0.525,y-0.08,S.centerZ);
  for(let level=0;level<levels.length-1;level++) {
    const bottom=levels[level],rise=levels[level+1]-bottom;
    const count=Math.ceil(rise/0.18/2)*2,straight=(count-S.winders)/2;
    const riser=rise/count,going=(S.turnX-S.startX)/straight;
    const outside:THREE.Vector3[]=[],inside:THREE.Vector3[]=[];
    const guard=(p:THREE.Vector3,tangent:THREE.Vector3,index:number)=>{
      const normal=new THREE.Vector3(-tangent.z,0,tangent.x).normalize();
      for(const side of [-1,1]) {
        const q=p.clone().addScaledVector(normal,side*S.width/2);
        posts.push(new THREE.Matrix4().compose(new THREE.Vector3(q.x,q.y+0.5,q.z),new THREE.Quaternion(),
new THREE.Vector3(1,1,1)));
        (side===-1?outside:inside).push(q.clone().add(new THREE.Vector3(0,1.02,0)));
        const winding=index>=straight&&index<straight+S.winders;
        const segments=winding?4:1;
        for(let segment=0;segment<segments;segment++) {
          const angle=(index-straight+(segment+0.5)/segments)*Math.PI/S.winders;
          const r=S.radius-side*S.width/2;
          const c=winding?new THREE.Vector3(S.turnX+r*Math.sin(angle),q.y,S.centerZ-r*Math.cos(angle)):q;
          markCameraCollider(root,{id:`east-service-${level}-${index}-${side}-${segment}`,shape:'box',
            center:{x:c.x,y:c.y+1,z:c.z},size:{x:0.06,y:2,z:winding?r*Math.PI/S.winders/segments+0.005:go
ing+0.015},
            rotationY:winding?Math.PI/2-angle:Math.atan2(tangent.x,tangent.z)});
        }
      }
      route.push(p);
    };
    for(let i=0;i<straight;i++) {
      const y=bottom+(i+1)*riser,x=S.startX+(i+0.5)*going,z=S.centerZ-S.radius;
      box('Service stair straight tread',going+0.012,0.055,S.width,x,y-0.0275,z);
      box('Service stair closed riser',0.025,riser,S.width,x-going/2,y-riser/2,z);
      guard(new THREE.Vector3(x,y,z),new THREE.Vector3(1,0,0),i);
    }
    // Eight genuinely curved wedge treads turn around a clear open well.
    for(let i=0;i<S.winders;i++) {
      const a=i*Math.PI/S.winders,b=(i+1)*Math.PI/S.winders;
      const shape=new THREE.Shape();
      for(const [radius,reverse] of [[S.radius+S.width/2,false],[S.radius-S.width/2,true]] as const) {
        for(let j=0;j<=8;j++) {
          const angle=reverse?b-(b-a)*j/8:a+(b-a)*j/8;
```

```
          const x=radius*Math.sin(angle),z=-radius*Math.cos(angle);
          if(!reverse&&j===0)shape.moveTo(x,z);else shape.lineTo(x,z);
        }
      }
      shape.closePath();const g=new THREE.ExtrudeGeometry(shape,{depth:riser,bevelEnabled:false});g.rotat
eX(Math.PI/2);
      const y=bottom+(straight+i+1)*riser;
      const tread=add('Service stair curved winder tread',g,materials.woodWarm,surfaces);tread.position.s
et(S.turnX,y,S.centerZ);
      const mid=(a+b)/2;
      guard(new THREE.Vector3(S.turnX+S.radius*Math.sin(mid),y,S.centerZ-S.radius*Math.cos(mid)),
        new THREE.Vector3(Math.cos(mid),0,Math.sin(mid)),straight+i);
    }
    for(let i=0;i<straight;i++) {
      const y=bottom+(straight+S.winders+i+1)*riser,x=S.turnX-(i+0.5)*going,z=S.centerZ+S.radius;
      box('Service stair straight tread',going+0.012,0.055,S.width,x,y-0.0275,z);
      box('Service stair closed riser',0.025,riser,S.width,x+going/2,y-riser/2,z);
      guard(new THREE.Vector3(x,y,z),new THREE.Vector3(-1,0,0),straight+S.winders+i);
    }
    rails(outside);rails(inside);
    for(const edge of [outside,inside]) {
      const vertices:number[]=[],indices:number[]=[];
      edge.forEach((p,i)=>{
        const direction=edge[Math.min(i+1,edge.length-1)].clone().sub(edge[Math.max(0,i-1)]);
        const n=new THREE.Vector3(-direction.z,0,direction.x).normalize().multiplyScalar(0.035);
        for(const [side,drop] of [[-1,1.12],[1,1.12],[1,1.34],[-1,1.34]])
          vertices.push(p.x+side*n.x,p.y-drop,p.z+side*n.z);
        if(i>0)for(let face=0;face<4;face++) {
          const a=(i-1)*4+face,b=(i-1)*4+(face+1)%4,c=i*4+(face+1)%4,d=i*4+face;
          indices.push(a,b,d,b,c,d);
        }
      });
      const g=new THREE.BufferGeometry();g.setAttribute('position',new THREE.Float32BufferAttribute(verti
ces,3));
      g.setIndex(indices);g.computeVertexNormals();add('Service stair continuous timber stringer',g,mater
ials.woodWarm);
    }
    // Cross the common landing before starting the next storey.
    for(let j=0;j<=12;j++)route.push(new THREE.Vector3(S.startX-0.5,levels[level+1],S.centerZ+S.radius-2*
S.radius*j/12));
  }
  const balusters=new THREE.InstancedMesh(geometry,materials.iron,posts.length);balusters.name='Service s
tair slender vertical balusters';
  posts.forEach((matrix,i)=>balusters.setMatrixAt(i,matrix));balusters.computeBoundingBox();balusters.com
puteBoundingSphere();root.add(balusters);
  root.userData.walkingSamples=route.map(p=>({x:p.x,y:p.y,z:p.z}));
  return root;
}
package com.elexvx.bookkin.annotations;
import com.elexvx.bookkin.annotations.AnnotationRepository.Annotation;
import com.elexvx.bookkin.annotations.AnnotationRepository.AnnotationBookPage;
import com.elexvx.bookkin.annotations.AnnotationRepository.AnnotationStyle;
import com.elexvx.bookkin.annotations.AnnotationRepository.AnnotationType;
import com.elexvx.bookkin.annotations.AnnotationExportService.ExportFormat;
import com.elexvx.bookkin.catalog.BookRepository;
import com.elexvx.bookkin.common.ApiException;
import com.elexvx.bookkin.users.UserRepository;
```

```
import jakarta.validation.Valid;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Size;
import java.security.Principal;
import java.util.List;
import java.util.UUID;
import java.nio.charset.StandardCharsets;
import org.springframework.context.annotation.Profile;
import org.springframework.http.ContentDisposition;
import org.springframework.http.HttpHeaders;
import org.springframework.http.MediaType;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
@RestController
@Profile("api")
@RequestMapping("/api/v1/annotations")
public class AnnotationController {
    private final AnnotationRepository annotations;
    private final UserRepository users;
    private final BookRepository books;
    private final AnnotationExportService exportService;
    public AnnotationController(AnnotationRepository annotations, UserRepository users, BookRepository bo
oks,
                                AnnotationExportService exportService) {
        this.annotations = annotations;
        this.users = users;
        this.books = books;
        this.exportService = exportService;
    }
    @GetMapping
    AnnotationList list(@RequestParam(required = false) UUID bookId, Principal principal) {
        return new AnnotationList(annotations.list(userId(principal), bookId));
    }
    @GetMapping("/books")
    AnnotationBookPage listBooks(@RequestParam(required = false) String q,
                                 @RequestParam(required = false) String cursor,
                                 @RequestParam(defaultValue = "36") int limit,
                                 Principal principal) {
        return annotations.listBooks(userId(principal), q, cursor, limit);
    }
    @GetMapping("/books/{bookId}/export/{format}")
    ResponseEntity<byte[]> export(@PathVariable UUID bookId, @PathVariable ExportFormat format, Principal
 principal) {
        var items = annotations.list(userId(principal), bookId).stream()
```

```
                .filter(annotation -> annotation.type() != AnnotationType.BOOKMARK).toList();
        if (items.isEmpty()) throw ApiException.notFound("ANNOTATIONS_NOT_FOUND", "这本书还没有可导出的笔
记或划线。");
        var file = exportService.export(items.getFirst().bookTitle(), items.getFirst().bookAuthor(), item
s, format);
        return ResponseEntity.ok()
                .contentType(MediaType.parseMediaType(file.mediaType()))
                .header(HttpHeaders.CONTENT_DISPOSITION, ContentDisposition.attachment()
                        .filename(file.filename(), StandardCharsets.UTF_8).build().toString())
                .contentLength(file.content().length)
                .body(file.content());
    }
    @PostMapping
    Annotation create(@Valid @RequestBody CreateAnnotation input, Principal principal) {
        books.findPreferredFile(input.bookId()).orElseThrow(() -> ApiException.notFound("BOOK_NOT_FOUND",
 "未找到这本书。"));
        return annotations.create(userId(principal), input.bookId(), input.type(), input.locator(), input
.quote(),
                input.note(), input.style(), input.color());
    }
    @DeleteMapping("/{id}")
    void delete(@PathVariable UUID id, Principal principal) {
        annotations.delete(userId(principal), id);
    }
    private UUID userId(Principal principal) {
        return users.findByUsername(principal.getName()).orElseThrow().id();
    }
    public record CreateAnnotation(@NotNull UUID bookId, @NotNull AnnotationType type,
                                   @NotBlank @Size(max = 4000) String locator,
                                   @Size(max = 10_000) String quote,
                                   @Size(max = 20_000) String note,
                                   @NotNull AnnotationStyle style,
                                   @Size(max = 16) String color) {}
    public record AnnotationList(List<Annotation> items) {}
}
package com.elexvx.bookkin.audit;
import org.springframework.context.annotation.Profile;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.web.bind.annotation.*;
@RestController
@Profile("api")
@RequestMapping("/api/v1/users/login-logs")
@PreAuthorize("hasAnyRole('OWNER','ADMIN')")
public class LoginLogController {
 private final AuditService audit;
 public LoginLogController(AuditService audit) { this.audit = audit; }
 @GetMapping
 public Page list(@RequestParam(defaultValue="0") int page, @RequestParam(defaultValue="20") int size) {
  return new Page(audit.loginEntries(Math.max(0, Math.min(page, 1000000)), Math.max(1, Math.min(size, 100
))), audit.loginCount());
 }
 public record Page(java.util.List<AuditService.LoginEntry> items, long total) {}
}
package com.elexvx.bookkin.auth;
import com.elexvx.bookkin.audit.AuditService;
```

```
import com.elexvx.bookkin.common.ApiException;
import com.elexvx.bookkin.users.BookKinUser;
import com.elexvx.bookkin.users.UserRepository;
import com.elexvx.bookkin.users.UserRole;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.validation.Valid;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Pattern;
import jakarta.validation.constraints.Size;
import java.net.InetAddress;
import java.net.UnknownHostException;
import java.security.Principal;
import java.util.Locale;
import org.springframework.context.annotation.Profile;
import org.springframework.http.HttpStatus;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.AuthenticationException;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.authentication.logout.SecurityContextLogoutHandler;
import org.springframework.security.web.context.HttpSessionSecurityContextRepository;
import org.springframework.security.web.csrf.CsrfToken;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
@RestController
@Profile("api")
@RequestMapping("/api/v1/auth")
public class AuthController {
    private final UserRepository users;
    private final PasswordEncoder passwords;
    private final AuthenticationManager authenticationManager;
    private final HttpSessionSecurityContextRepository contextRepository;
    private final LoginRateLimiter rateLimiter;
    private final AuditService audit;
    private final com.elexvx.bookkin.auth.PasswordPolicyService passwordPolicy;
    public AuthController(UserRepository users, PasswordEncoder passwords, AuthenticationManager authenti
cationManager,
                          HttpSessionSecurityContextRepository contextRepository, LoginRateLimiter rateLi
miter,
                          AuditService audit, com.elexvx.bookkin.auth.PasswordPolicyService passwordPolic
y) {
        this.users = users;
        this.passwords = passwords;
        this.authenticationManager = authenticationManager;
        this.contextRepository = contextRepository;
        this.rateLimiter = rateLimiter;
        this.audit = audit; this.passwordPolicy=passwordPolicy;
```

```
    }
    @GetMapping("/csrf")
    CsrfResponse csrf(CsrfToken token) {
        return new CsrfResponse(token.getToken(), token.getHeaderName());
    }
    @GetMapping("/setup-status")
    SetupStatus setupStatus() {
        return new SetupStatus(users.count() > 0);
    }
    @PostMapping("/setup")
    SessionUser setup(@Valid @RequestBody SetupRequest input, HttpServletRequest request, HttpServletResp
onse response) {
        if (!isTrustedSetupAddress(request.getRemoteAddr())) {
            throw ApiException.forbidden("SETUP_NETWORK_FORBIDDEN", "主人初始化只允许从 NAS 本机或局域网
访问。");
        }
        passwordPolicy.validate(input.password());
        var created = users.createOwnerIfEmpty(input.username(), input.displayName(), passwords.encode(in
put.password()))
                .orElseThrow(() -> ApiException.conflict("ALREADY_INITIALIZED", "BookKin已经完成初始化。"
));
        authenticateAndSave(created.username(), input.password(), request, response);
        audit.record(created.id(), "OWNER_INITIALIZED", "USER", created.id().toString(), null, null, null
, null,
                "SUCCEEDED", "{}");
        return SessionUser.from(created);
    }
    @PostMapping("/login")
    SessionUser login(@Valid @RequestBody LoginRequest input, HttpServletRequest request, HttpServletResp
onse response) {
        String key = request.getRemoteAddr() + ":" + input.username().toLowerCase(Locale.ROOT);
        rateLimiter.check(key);
        try {
            authenticateAndSave(input.username(), input.password(), request, response);
            var user = users.findByUsername(input.username()).orElseThrow();
            users.recordLogin(user.id());
            rateLimiter.success(key);
            audit.recordLogin(user.id(), input.username(), "SUCCEEDED", request.getRemoteAddr());
            return SessionUser.from(user);
        } catch (AuthenticationException exception) {
            rateLimiter.failure(key);
            audit.recordLogin(users.findByUsername(input.username()).map(BookKinUser::id).orElse(null),
                    input.username().toLowerCase(Locale.ROOT), "FAILED", request.getRemoteAddr());
            throw new ApiException(HttpStatus.UNAUTHORIZED, "BAD_CREDENTIALS", "用户名或密码不正确。");
        }
    }
    @GetMapping("/session")
    SessionUser session(Principal principal) {
        return SessionUser.from(current(principal));
    }
    @PutMapping("/password")
    SessionUser changePassword(@Valid @RequestBody ChangePasswordRequest input, Principal principal) {
        var user = current(principal);
        if (!passwords.matches(input.currentPassword(), user.passwordHash())) {
            throw new ApiException(HttpStatus.BAD_REQUEST, "CURRENT_PASSWORD_INVALID", "当前密码不正确。"
);
        }
```

```
        passwordPolicy.validate(input.newPassword());
        users.updatePassword(user.id(), passwords.encode(input.newPassword()));
        audit.record(user.id(), "PASSWORD_CHANGED", "USER", user.id().toString(), null, null, null, null,
                "SUCCEEDED", "{}");
        return SessionUser.from(users.findById(user.id()).orElseThrow());
    }
    @PostMapping("/logout")
    void logout(HttpServletRequest request, HttpServletResponse response) {
        var authentication = SecurityContextHolder.getContext().getAuthentication();
        var user = authentication == null ? null : users.findByUsername(authentication.getName()).orElse(
null);
        new SecurityContextLogoutHandler().logout(request, response, authentication);
        if (user != null) audit.record(user.id(), "LOGOUT", "USER", user.id().toString(), null, null, nul
l, null,
                "SUCCEEDED", "{}");
    }
    private BookKinUser current(Principal principal) {
        if (principal == null) throw new ApiException(HttpStatus.UNAUTHORIZED, "AUTH_REQUIRED", "请先登录
。");
        return users.findByUsername(principal.getName()).orElseThrow();
    }
    private void authenticateAndSave(String username, String password, HttpServletRequest request, HttpSe
rvletResponse response) {
        var authentication = authenticationManager.authenticate(UsernamePasswordAuthenticationToken.unaut
henticated(username, password));
        request.getSession(true);
        request.changeSessionId();
        var context = SecurityContextHolder.createEmptyContext();
        context.setAuthentication(authentication);
        SecurityContextHolder.setContext(context);
        contextRepository.saveContext(context, request, response);
    }
    static boolean isTrustedSetupAddress(String remoteAddress) {
        if (remoteAddress == null) return false;
        String literal = remoteAddress.contains("%") ? remoteAddress.substring(0, remoteAddress.indexOf('
%')) : remoteAddress;
        if (!literal.matches("[0-9a-fA-F:.]+")) return false;
        try {
            InetAddress address = InetAddress.getByName(literal);
            if (address.isAnyLocalAddress() || address.isLoopbackAddress()
                    || address.isLinkLocalAddress() || address.isSiteLocalAddress()) return true;
            byte[] bytes = address.getAddress();
            return bytes.length == 16 && (bytes[0] & 0xfe) == 0xfc;
        } catch (UnknownHostException exception) {
            return false;
        }
    }
    public record CsrfResponse(String token, String headerName) {}
    public record SetupStatus(boolean initialized) {}
    public record LoginRequest(@NotBlank String username, @NotBlank String password) {}
    public record SetupRequest(
            @NotBlank @Pattern(regexp = "[a-zA-Z0-9._-]{3,80}") String username,
            @NotBlank @Size(max = 120) String displayName,
            @NotBlank @Size(max = 200) String password) {}
    public record ChangePasswordRequest(@NotBlank String currentPassword, @NotBlank @Size(max = 200) Stri
ng newPassword) {}
    public record SessionUser(String id, String username, String displayName, UserRole role, boolean must
ChangePassword) {
```

```
        static SessionUser from(BookKinUser user) {
            return new SessionUser(user.id().toString(), user.username(), user.displayName(), user.role()
, user.mustChangePassword());
        }
    }
}
import {LONG_ROOM} from '../longRoomLayout';
type Point={x:number;y:number;z:number};
/** Use the stair builder's own samples, so the guided camera follows its treads. */
export function createServiceWalk(samples:readonly Point[]) {
 const points=samples.map(p=>({...p,z:p.z+LONG_ROOM.length/2}));
 if(points.length<2)throw new Error('Service stair route requires tread samples');
 const lengths=points.slice(1).map((p,i)=>Math.hypot(p.x-points[i].x,p.y-points[i].y,p.z-points[i].z));
 const length=lengths.reduce((a,b)=>a+b,0);
 return {length,pointAt(distance:number):Point {
  let remaining=Math.max(0,Math.min(distance,length));
  for(let i=0;i<lengths.length;i++) {
   if(remaining<=lengths[i]||i===lengths.length-1) {
    const a=points[i],b=points[i+1],t=lengths[i]===0?0:remaining/lengths[i];
    return {x:a.x+(b.x-a.x)*t,y:a.y+(b.y-a.y)*t,z:a.z+(b.z-a.z)*t};
   }
   remaining-=lengths[i];
  }
  return {...points[points.length-1]};
 }};
}
export default {
  async fetch(request, env) {
    const response = await env.ASSETS.fetch(request);
    const acceptsHtml = request.headers.get("accept")?.includes("text/html");
    if (response.status !== 404 || !acceptsHtml || !["GET", "HEAD"].includes(request.method)) {
      return response;
    }
    const indexUrl = new URL(request.url);
    indexUrl.pathname = "/index.html";
    indexUrl.search = "";
    return env.ASSETS.fetch(new Request(indexUrl, request));
  },
};
import { Box } from "@/ui/primitives";
import { Stack } from "@/ui/primitives";
import { Typography } from "@/ui/primitives";
import type { ReactElement, ReactNode } from "react";
import { tokens } from "../theme/generated-tokens";
export function OverviewCardHeader({
  id,
  icon,
  children,
  inverse = false,
}: {
  id: string;
```

```
  icon: ReactElement;
  children: ReactNode;
  inverse?: boolean;
}) {
  return (
    <Stack
      direction="row"
      sx={{
        minHeight: `${tokens.typography.fontSize.title}px`,
        alignItems: "center",
        gap: `${tokens.spacing[2]}px`,
        // The progress card is a dark surface in every site theme.  Keep its
        // heading/icon at the same size and rhythm as the other cards while
        // switching to the high-contrast text token instead of a theme
        // primary tint that becomes muddy in Night mode.
        color: inverse ? "common.white" : "primary.main",
      }}
    >
      <Box
        component="span"
        aria-hidden="true"
        sx={{
          display: "inline-flex",
          flexShrink: 0,
          "& .bk-icon": { fontSize: `${tokens.typography.fontSize.title}px` },
        }}
      >
        {icon}
      </Box>
      <Typography
        id={id}
        variant="h6"
        component="h2"
        sx={{ fontWeight: tokens.typography.fontWeight.semibold }}
      >
        {children}
      </Typography>
    </Stack>
  );
}
import { Box } from "@/ui/primitives";
import { Stack } from "@/ui/primitives";
import { Typography } from "@/ui/primitives";
import type { ReactNode } from "react";
import { tokens } from "../theme/generated-tokens";
export function OverviewEmptyState({
  title,
  description,
  titleId,
  inverse = false,
```

```
}: {
  title: ReactNode;
  description: ReactNode;
  titleId?: string;
  inverse?: boolean;
}) {
  return (
    <Stack
      sx={{
        minHeight: `${tokens.spacing[20]}px`,
        alignItems: "flex-start",
        justifyContent: "center",
        mt: `${tokens.spacing[4]}px`,
      }}
    >
      <Box>
        <Typography id={titleId} variant="h4">{title}</Typography>
        <Typography
          variant="body2"
          color={inverse ? "inherit" : "text.secondary"}
          sx={{ mt: `${tokens.spacing[1]}px`, ...(inverse ? { opacity: 0.68 } : {}) }}
        >
          {description}
        </Typography>
      </Box>
    </Stack>
  );
}
package com.elexvx.bookkin.auth;
import jakarta.validation.Valid;
import java.security.Principal;
import org.springframework.context.annotation.Profile;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.web.bind.annotation.*;
@RestController
@Profile("api")
@RequestMapping("/api/v1")
public class PasswordPolicyController {
 private final PasswordPolicyService service;
 public PasswordPolicyController(PasswordPolicyService service) { this.service=service; }
 @GetMapping("/auth/password-policy") public PasswordPolicyService.Policy policy() {return service.policy
();}
 @PutMapping("/users/password-policy") @PreAuthorize("hasRole('OWNER')")
 public PasswordPolicyService.Policy save(@Valid @RequestBody PasswordPolicyService.Policy input,Principa
l principal) {return service.save(principal.getName(),input);}
}
package com.elexvx.bookkin.auth;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.validation.Valid;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Pattern;
```

```
import jakarta.validation.constraints.Size;
import java.security.Principal;
import org.springframework.context.annotation.Profile;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.web.context.HttpSessionSecurityContextRepository;
import org.springframework.web.bind.annotation.*;
@RestController
@Profile("api")
@RequestMapping("/api/v1/auth/profile")
public class ProfileController {
    private final ProfileService profiles;
    private final AuthenticationManager authentication;
    private final HttpSessionSecurityContextRepository contexts;
    private final LoginRateLimiter limiter;
    public ProfileController(ProfileService profiles, AuthenticationManager authentication,
            HttpSessionSecurityContextRepository contexts, LoginRateLimiter limiter) {
        this.profiles=profiles; this.authentication=authentication; this.contexts=contexts; this.limiter=
limiter;
    }
    @PutMapping
    AuthController.SessionUser update(@Valid @RequestBody UpdateProfile input, Principal principal,
            HttpServletRequest request, HttpServletResponse response) {
        String key="profile:"+principal.getName();
        limiter.check(key);
        com.elexvx.bookkin.users.BookKinUser user;
        try { user=profiles.update(principal.getName(), input.username(), input.displayName(), input.curr
entPassword()); }
        catch (com.elexvx.bookkin.common.ApiException error) { limiter.failure(key); throw error; }
        limiter.success(key);
        if (request.getSession(false) != null) request.getSession(false).invalidate();
        var auth=authentication.authenticate(UsernamePasswordAuthenticationToken.unauthenticated(user.use
rname(), input.currentPassword()));
        var context=SecurityContextHolder.createEmptyContext(); context.setAuthentication(auth);
        SecurityContextHolder.setContext(context); contexts.saveContext(context, request, response);
        return AuthController.SessionUser.from(user);
    }
    public record UpdateProfile(
        @NotBlank @Pattern(regexp="[a-zA-Z0-9._-]{3,80}") String username,
        @NotBlank @Size(max=120) String displayName,
        @NotBlank @Size(max=200) String currentPassword) {}
}
import { Box } from "@/ui/primitives";
import { Stack } from "@/ui/primitives";
import { Typography } from "@/ui/primitives";
import type { SxProps, Theme } from "@/ui/primitives";
import { createContext, useContext, type ReactNode } from "react";
export const EmbeddedSettingsContext = createContext(false);
import { tokens } from "../theme/generated-tokens";
export const pageWidthSx = {
  width: "100%",
  maxWidth: tokens.layout.contentMax,
```

```
  mx: "auto",
  px: { xs: 2, sm: 3, lg: 0 },
};
// Reader chrome and document surfaces share the exact page frame at every
// viewport width. A narrower intermediate reader frame makes the two stacked
// navigation bars visibly drift apart on tablets and small laptops.
export const readerWidthSx = pageWidthSx;
export const topNavigationSurfaceSx = {
  bgcolor: "background.default",
  color: "text.primary",
  borderBottom: 0,
  boxShadow: "none",
};
export function PageContainer({ children, sx }: { children: ReactNode; sx?: SxProps<Theme> }) {
  const embedded = useContext(EmbeddedSettingsContext);
  if (embedded) return <Box sx={sx}>{children}</Box>;
  return (
    <Box
      sx={{
        ...pageWidthSx,
        pt: { xs: 4, sm: 6, md: 8 },
        pb: { xs: 6, md: 8 },
        ...sx,
      }}
    >
      {children}
    </Box>
  );
}
export function PageHeader({
  eyebrow,
  title,
  description,
  action,
}: {
  eyebrow?: string;
  title: string;
  description?: string;
  action?: ReactNode;
}) {
  const embedded = useContext(EmbeddedSettingsContext);
  return (
    <Stack
      direction={{ xs: "column", sm: "row" }}
      sx={{
        alignItems: { xs: "stretch", sm: "flex-end" },
        justifyContent: "space-between",
        gap: embedded ? 2 : 3,
        mb: action || embedded ? 3 : 4,
      }}
```

```
    >
      <Box sx={{ minWidth: 0 }}>
        {eyebrow && !embedded && <Typography variant="overline" color="text.secondary" sx={{ fontWeight:
tokens.typography.fontWeight.medium }}>{eyebrow}</Typography>}
        <Typography variant={embedded ? "h5" : "h1"} component={embedded ? "h2" : "h1"} sx={embedded ? un
defined : { fontSize: { xs: 32, sm: tokens.typography.fontSize.displayLg } }}>{title}</Typography>
        {description && <Typography color="text.secondary" sx={{ mt: 1.5, maxWidth: 720 }}>{description}<
/Typography>}
      </Box>
      {action && <ActionToolbar>{action}</ActionToolbar>}
    </Stack>
  );
}
export function ActionToolbar({ children }: { children: ReactNode }) {
  return <Stack className="bk-action-toolbar" direction="row" sx={{ width: { xs: "100%", sm: "auto" }, ju
stifyContent: { xs: "flex-start", sm: "flex-end" }, alignItems: "center", flexWrap: "wrap", gap: 1.5 }}>{
children}</Stack>;
}
export function FormActions({ children }: { children: ReactNode }) {
  return <Stack direction="row" sx={{ width: "100%", justifyContent: "flex-end" }}>{children}</Stack>;
}
package com.elexvx.bookkin.auth;
import com.elexvx.bookkin.users.UserRepository;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.validation.Valid;
import jakarta.validation.constraints.*;
import java.security.Principal;
import java.util.Map;
import org.springframework.context.annotation.Profile;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.web.bind.annotation.*;
@RestController
@Profile("api")
@RequestMapping("/api/v1")
public class RecoveryController {
    private final MailService mail;
    private final RecoveryService recovery;
    private final UserRepository users;
    private final LoginRateLimiter limiter;
    public RecoveryController(MailService mail, RecoveryService recovery, UserRepository users, LoginRate
Limiter limiter) {
        this.mail=mail; this.recovery=recovery; this.users=users; this.limiter=limiter;
    }
    @GetMapping("/mail/settings") @PreAuthorize("hasRole('OWNER')")
    MailService.Settings settings() { return mail.settings(); }
    @PutMapping("/mail/settings") @PreAuthorize("hasRole('OWNER')")
    MailService.Settings save(@Valid @RequestBody MailService.Settings input) { return mail.save(input);
}
    @PostMapping("/mail/test") @PreAuthorize("hasRole('OWNER')")
    void test(@Valid @RequestBody EmailInput input, HttpServletRequest req) {
        limit("test:"+req.getRemoteAddr()); mail.send(input.email(),"BookKin 测试邮件","邮件服务已连接成
功。你可以使用此服务发送邮箱验证和密码重置链接。");
    }
    @GetMapping("/auth/recovery-email")
    Map<String,String> email(Principal p) { return Map.of("email",recovery.email(users.findByUsername(p.g
etName()).orElseThrow().id())); }
    @PostMapping("/auth/recovery-email")
    void bind(@Valid @RequestBody BindInput input, Principal p, HttpServletRequest req) {
        limit("bind:"+req.getRemoteAddr()); recovery.bind(users.findByUsername(p.getName()).orElseThrow()
,input.email(),input.password());
```

```
    }
    @PostMapping("/auth/recovery/request")
    void request(@Valid @RequestBody EmailInput input, HttpServletRequest req) {
        limit("request-ip:"+req.getRemoteAddr()); limit("request-email:"+input.email().trim().toLowerCase
(java.util.Locale.ROOT)); recovery.request(input.email());
    }
    @PostMapping("/auth/recovery/verify")
    void verify(@Valid @RequestBody TokenInput input, HttpServletRequest req) {
        limit("verify:"+req.getRemoteAddr()); recovery.consume(input.token(),"BIND",null);
    }
    @PostMapping("/auth/recovery/reset")
    void reset(@Valid @RequestBody ResetInput input, HttpServletRequest req) {
        limit("reset:"+req.getRemoteAddr()); recovery.consume(input.token(),"RESET",input.password());
    }
    private void limit(String key) { limiter.consumeRecovery(key); }
    public record EmailInput(@NotBlank @Email @Size(max=320) String email) {}
    public record BindInput(@NotBlank @Email @Size(max=320) String email,@NotBlank @Size(max=200) String
password) {}
    public record TokenInput(@NotBlank @Size(max=100) String token) {}
    public record ResetInput(@NotBlank @Size(max=100) String token,@NotBlank @Size(max=200) String passwo
rd) {}
}
package com.elexvx.bookkin.auth;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.validation.Valid;
import jakarta.validation.constraints.*;
import java.security.Principal;
import org.springframework.context.annotation.Profile;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.web.bind.annotation.*;
@RestController
@Profile("api")
@RequestMapping("/api/v1")
public class RegistrationController {
 private final RegistrationService service;
 private final LoginRateLimiter limiter;
 public RegistrationController(RegistrationService service, LoginRateLimiter limiter) { this.service=serv
ice; this.limiter=limiter; }
 @GetMapping("/auth/registration-status")
 public Policy status() { return new Policy(service.enabled()); }
 @GetMapping("/users/security-settings")
 @PreAuthorize("hasAnyRole('OWNER','ADMIN')")
 public Policy settings() { return status(); }
 @PutMapping("/users/security-settings")
 @PreAuthorize("hasRole('OWNER')")
 public Policy configure(@Valid @RequestBody Policy input, Principal principal) { service.configure(princ
ipal.getName(),input.registrationEnabled()); return status(); }
 @PostMapping("/auth/register")
 public Registered register(@Valid @RequestBody Registration input, HttpServletRequest request) {
  limiter.consumeRegistration(request.getRemoteAddr());
  service.register(input.username().trim(),input.displayName().trim(),input.password());
  return new Registered(true);
 }
 public record Registered(boolean registered) {}
 public record Policy(@NotNull Boolean registrationEnabled) {}
```

```
 public record Registration(@NotBlank @Pattern(regexp="[a-zA-Z0-9._-]{3,80}") String username, @NotBlank
@Size(max=120) String displayName, @NotBlank @Size(max=200) String password) {}
}
package com.elexvx.bookkin.catalog;
import com.elexvx.bookkin.common.ApiException;
import com.elexvx.bookkin.users.UserRepository;
import java.security.Principal;
import java.nio.file.LinkOption;
import java.nio.file.Path;
import java.util.UUID;
import org.springframework.core.io.FileSystemResource;
import org.springframework.http.CacheControl;
import org.springframework.http.MediaType;
import org.springframework.http.ResponseEntity;
import org.springframework.context.annotation.Profile;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
@RestController
@Profile("api")
@RequestMapping("/api/v1/books")
public class BookController {
    private final BookRepository books;
    private final UserRepository users;
    public BookController(BookRepository books, UserRepository users) {
        this.books = books;
        this.users = users;
    }
    @GetMapping
    BookRepository.BookPage list(@RequestParam(required = false) String q,
                                 @RequestParam(required = false) BookFormat format,
                                 @RequestParam(defaultValue = "RECENT") BookSort sort,
                                 @RequestParam(required = false) String cursor,
                                 @RequestParam(defaultValue = "60") int limit,
                                 Principal principal) {
        return books.list(userId(principal), q, format, sort, cursor, limit);
    }
    @GetMapping("/revision")
    BookRepository.CatalogRevision revision(Principal principal) {
        return books.catalogRevision(userId(principal));
    }
    @GetMapping("/{id}")
    BookRepository.BookSummary get(@PathVariable UUID id, Principal principal) {
        return books.findSummary(userId(principal), id)
                .orElseThrow(() -> ApiException.notFound("BOOK_NOT_FOUND", "未找到这本书。"));
    }
    @GetMapping("/{id}/cover")
    ResponseEntity<FileSystemResource> cover(@PathVariable UUID id) {
        var location = books.findCover(id).orElseThrow(() -> ApiException.notFound("COVER_NOT_FOUND", "尚
未生成封面。"));
```

```
        Path root;
        Path cover;
        try {
            root = Path.of(location.canonicalRoot() == null ? location.configuredRoot() : location.canoni
calRoot()).toRealPath();
            Path candidate = root.resolve(location.relativePath()).normalize();
            if (!candidate.startsWith(root) || java.nio.file.Files.isSymbolicLink(candidate)
                    || !java.nio.file.Files.isRegularFile(candidate, LinkOption.NOFOLLOW_LINKS)) {
                throw ApiException.notFound("COVER_NOT_FOUND", "封面缓存不存在。");
            }
            cover = candidate.toRealPath();
            if (!cover.startsWith(root)) throw ApiException.notFound("COVER_NOT_FOUND", "封面缓存不存在。
");
        } catch (java.io.IOException exception) {
            throw ApiException.notFound("COVER_NOT_FOUND", "封面缓存不存在。");
        }
        String lower = cover.getFileName().toString().toLowerCase(java.util.Locale.ROOT);
        MediaType type = lower.endsWith(".png") ? MediaType.IMAGE_PNG
                : lower.endsWith(".webp") ? MediaType.parseMediaType("image/webp") : MediaType.IMAGE_JPEG
;
        return ResponseEntity.ok().contentType(type).cacheControl(CacheControl.maxAge(java.time.Duration.
ofDays(30)).cachePublic())
                .body(new FileSystemResource(cover));
    }
    private UUID userId(Principal principal) {
        return users.findByUsername(principal.getName()).orElseThrow().id();
    }
}
import {
  AmbientLight,
  BackSide,
  CanvasTexture,
  Color,
  DirectionalLight,
  FrontSide,
  Mesh,
  MeshStandardMaterial,
  OrthographicCamera,
  PCFShadowMap,
  PlaneGeometry,
  RepeatWrapping,
  Scene,
  ShadowMaterial,
  SRGBColorSpace,
  WebGLRenderer,
} from "three";
import { tokens } from "../../theme/generated-tokens";
import {
  PAGE_CURL_HORIZONTAL_SEGMENTS,
  PAGE_CURL_VERTICAL_SEGMENTS,
  pageCurlPath,
  pageCurlPointForColumn,
  type PageTurnDirection,
} from "./page-turn-geometry";
```

```
import { pageTurnPixelRatio } from "./page-turn-rendering";
const PAGE_CAMERA_Z = 6;
const PAGE_SHADOW_OPACITY = 0.22;
const PAGE_TEXTURE_FADE_START = 0.12;
const PAGE_TEXTURE_FADE_END = 0.48;
const PAGE_TEXTURE_MID_TURN_OPACITY = 0.12;
export type PageCurlRenderer = {
  render: (progress: number) => void;
  setBackSnapshot: (snapshot: HTMLCanvasElement) => void;
  dispose: () => void;
};
type PageTextureRegion = {
  start: number;
  width: number;
};
export type PageCurlTextureLayout = {
  turning: PageTextureRegion;
  back: PageTextureRegion;
  static?: PageTextureRegion;
};
const FULL_PAGE_TEXTURE: PageTextureRegion = { start: 0, width: 1 };
const clampUnit = (value: number) => Math.min(1, Math.max(0, value));
export function pageTextureFacingOpacity(facing: number) {
  const position = clampUnit(
    (Math.abs(facing) - PAGE_TEXTURE_FADE_START)
    / (PAGE_TEXTURE_FADE_END - PAGE_TEXTURE_FADE_START),
  );
  return position * position * (3 - 2 * position);
}
export function pageTextureMotionOpacity(progress: number) {
  const distanceFromMiddle = Math.abs(2 * clampUnit(progress) - 1);
  const easedDistance = distanceFromMiddle * distanceFromMiddle
    * (3 - 2 * distanceFromMiddle);
  return PAGE_TEXTURE_MID_TURN_OPACITY
    + (1 - PAGE_TEXTURE_MID_TURN_OPACITY) * easedDistance;
}
export function pageCurlTextureLayout(direction: PageTurnDirection, spread: boolean): PageCurlTextureLayo
ut {
  if (!spread) return { turning: FULL_PAGE_TEXTURE, back: FULL_PAGE_TEXTURE };
  return direction === "NEXT"
    ? {
        turning: { start: 0.5, width: 0.5 },
        back: { start: 0, width: 0.5 },
        static: { start: 0, width: 0.5 },
      }
    : {
        turning: { start: 0, width: 0.5 },
        back: { start: 0.5, width: 0.5 },
        static: { start: 0.5, width: 0.5 },
      };
}
```

```
function pageTexture(
  renderer: WebGLRenderer,
  source: HTMLCanvasElement,
  region: PageTextureRegion,
  mirrored = false,
) {
  const texture = new CanvasTexture(source);
  texture.colorSpace = SRGBColorSpace;
  texture.anisotropy = Math.min(8, renderer.capabilities.getMaxAnisotropy());
  if (mirrored || region.width !== 1 || region.start !== 0) {
    texture.wrapS = RepeatWrapping;
  }
  texture.repeat.x = (mirrored ? -1 : 1) * region.width;
  texture.offset.x = mirrored ? region.start + region.width : region.start;
  texture.needsUpdate = true;
  return texture;
}
function pageBackgroundColor(source: HTMLCanvasElement) {
  try {
    const pixel = source.getContext("2d", { willReadFrequently: true })
      ?.getImageData(0, 0, 1, 1).data;
    if (pixel) return new Color(`rgb(${pixel[0]}, ${pixel[1]}, ${pixel[2]})`);
  } catch {
    // Cross-origin images can make a snapshot unreadable. The semantic canvas
    // color is still a safe paper-color fallback for the edge-on fold.
  }
  return new Color(tokens.color.semantic.canvas);
}
function softenEdgeOnText(
  material: MeshStandardMaterial,
  source: HTMLCanvasElement,
  motionOpacityUniform: { value: number },
) {
  const backgroundUniform = { value: pageBackgroundColor(source) };
  material.customProgramCacheKey = () => "page-texture-facing-fade-v1";
  material.onBeforeCompile = (shader) => {
    shader.uniforms.pageTextureBackground = backgroundUniform;
    shader.uniforms.pageTextureMotionOpacity = motionOpacityUniform;
    shader.fragmentShader = shader.fragmentShader.replace(
      "#include <map_fragment>",
      `#include <map_fragment>
#ifndef FLAT_SHADED
  float pageTextureFacing = smoothstep(
    ${PAGE_TEXTURE_FADE_START.toFixed(2)},
    ${PAGE_TEXTURE_FADE_END.toFixed(2)},
    abs(normalize(vNormal).z)
  );
  diffuseColor.rgb = mix(
    pageTextureBackground,
    diffuseColor.rgb,
```

```
    pageTextureFacing * pageTextureMotionOpacity
  );
#endif`,
    ).replace(
      "#include <common>",
      `#include <common>
uniform vec3 pageTextureBackground;
uniform float pageTextureMotionOpacity;`,
    );
  };
  return backgroundUniform;
}
export function createPageCurlRenderer({
  canvas,
  frontSnapshot,
  direction,
  spread = false,
}: {
  canvas: HTMLCanvasElement;
  frontSnapshot: HTMLCanvasElement;
  direction: PageTurnDirection;
  spread?: boolean;
}): PageCurlRenderer {
  const bounds = canvas.getBoundingClientRect();
  if (bounds.width < 1 || bounds.height < 1) throw new Error("翻页画布没有可用尺寸");
  const renderer = new WebGLRenderer({
    canvas,
    alpha: true,
    antialias: true,
    powerPreference: "high-performance",
    premultipliedAlpha: true,
  });
  renderer.setPixelRatio(pageTurnPixelRatio(window.devicePixelRatio || 1));
  renderer.setSize(bounds.width, bounds.height, false);
  renderer.setClearColor(new Color(tokens.color.primitive.black), 0);
  renderer.outputColorSpace = SRGBColorSpace;
  renderer.shadowMap.enabled = true;
  renderer.shadowMap.type = PCFShadowMap;
  const aspect = bounds.width / bounds.height;
  const pageHeight = 2;
  const pageWidth = pageHeight * aspect;
  const turningPageWidth = spread ? pageWidth / 2 : pageWidth;
  const turningPageOffset = spread
    ? (direction === "NEXT" ? pageWidth / 4 : -pageWidth / 4)
    : 0;
  const staticPageOffset = -turningPageOffset;
  const textureLayout = pageCurlTextureLayout(direction, spread);
  const camera = new OrthographicCamera(
    -pageWidth / 2,
    pageWidth / 2,
```

```
    pageHeight / 2,
    -pageHeight / 2,
    0.1,
    20,
  );
  camera.position.set(0, 0, PAGE_CAMERA_Z);
  camera.lookAt(0, 0, 0);
  const scene = new Scene();
  const geometry = new PlaneGeometry(
    turningPageWidth,
    pageHeight,
    PAGE_CURL_HORIZONTAL_SEGMENTS,
    PAGE_CURL_VERTICAL_SEGMENTS,
  );
  const originalPositions = Float32Array.from(geometry.attributes.position.array as ArrayLike<number>);
  const frontTexture = pageTexture(renderer, frontSnapshot, textureLayout.turning);
  let backTexture = pageTexture(renderer, frontSnapshot, textureLayout.back, true);
  const frontMaterial = new MeshStandardMaterial({
    color: new Color(tokens.color.primitive.white),
    map: frontTexture,
    metalness: 0,
    roughness: 0.96,
    side: FrontSide,
  });
  const backMaterial = new MeshStandardMaterial({
    color: new Color(tokens.color.primitive.white),
    map: backTexture,
    metalness: 0,
    roughness: 1,
    side: BackSide,
  });
  const motionOpacityUniform = { value: 1 };
  softenEdgeOnText(frontMaterial, frontSnapshot, motionOpacityUniform);
  const backBackgroundUniform = softenEdgeOnText(
    backMaterial,
    frontSnapshot,
    motionOpacityUniform,
  );
  const frontPage = new Mesh(geometry, frontMaterial);
  const backPage = new Mesh(geometry, backMaterial);
  frontPage.position.set(turningPageOffset, 0, 0.002);
  backPage.position.copy(frontPage.position);
  frontPage.castShadow = true;
  backPage.castShadow = true;
  scene.add(frontPage, backPage);
  const staticTexture = textureLayout.static
    ? pageTexture(renderer, frontSnapshot, textureLayout.static)
    : null;
  const staticMaterial = staticTexture
    ? new MeshStandardMaterial({
```

```
        color: new Color(tokens.color.primitive.white),
        map: staticTexture,
        metalness: 0,
        roughness: 0.96,
        side: FrontSide,
      })
    : null;
  const staticGeometry = staticMaterial ? new PlaneGeometry(turningPageWidth, pageHeight) : null;
  if (staticGeometry && staticMaterial) {
    const staticPage = new Mesh(staticGeometry, staticMaterial);
    staticPage.position.x = staticPageOffset;
    scene.add(staticPage);
  }
  const shadowMaterial = new ShadowMaterial({
    color: new Color(tokens.color.primitive.black),
    opacity: PAGE_SHADOW_OPACITY,
    transparent: true,
  });
  const shadowReceiver = new Mesh(new PlaneGeometry(pageWidth, pageHeight), shadowMaterial);
  shadowReceiver.position.z = -0.035;
  shadowReceiver.receiveShadow = true;
  scene.add(shadowReceiver);
  scene.add(new AmbientLight(new Color(tokens.color.primitive.white), 2.1));
  const keyLight = new DirectionalLight(new Color(tokens.color.primitive.white), 1.15);
  keyLight.position.set(direction === "NEXT" ? 2.4 : -2.4, 2.8, 4.5);
  keyLight.castShadow = true;
  keyLight.shadow.mapSize.set(1024, 1024);
  keyLight.shadow.camera.left = -pageWidth * 1.6;
  keyLight.shadow.camera.right = pageWidth * 1.6;
  keyLight.shadow.camera.top = pageHeight * 1.4;
  keyLight.shadow.camera.bottom = -pageHeight * 1.4;
  scene.add(keyLight);
  const render = (progress: number) => {
    motionOpacityUniform.value = pageTextureMotionOpacity(progress);
    const positions = geometry.attributes.position;
    const rowLength = PAGE_CURL_HORIZONTAL_SEGMENTS + 1;
    for (let row = 0; row <= PAGE_CURL_VERTICAL_SEGMENTS; row += 1) {
      const rowStart = row * rowLength;
      const firstPositionIndex = rowStart * 3;
      const originalY = originalPositions[firstPositionIndex + 1];
      const rowPosition = (originalY + pageHeight / 2) / pageHeight;
      const path = pageCurlPath({
        progress,
        direction,
        width: turningPageWidth,
        segments: PAGE_CURL_HORIZONTAL_SEGMENTS,
        rowPosition,
      });
      for (let column = 0; column <= PAGE_CURL_HORIZONTAL_SEGMENTS; column += 1) {
        const vertexIndex = rowStart + column;
```

```
        const point = pageCurlPointForColumn(path, column, direction);
        positions.setXYZ(vertexIndex, point.x, originalY, point.z);
      }
    }
    positions.needsUpdate = true;
    geometry.computeVertexNormals();
    renderer.render(scene, camera);
  };
  return {
    render,
    setBackSnapshot(snapshot) {
      const nextTexture = pageTexture(renderer, snapshot, textureLayout.back, true);
      backMaterial.map = nextTexture;
      backMaterial.needsUpdate = true;
      backBackgroundUniform.value.copy(pageBackgroundColor(snapshot));
      backTexture.dispose();
      backTexture = nextTexture;
    },
    dispose() {
      frontTexture.dispose();
      backTexture.dispose();
      staticTexture?.dispose();
      frontMaterial.dispose();
      backMaterial.dispose();
      staticMaterial?.dispose();
      staticGeometry?.dispose();
      shadowMaterial.dispose();
      shadowReceiver.geometry.dispose();
      geometry.dispose();
      renderer.dispose();
    },
  };
}
package com.elexvx.bookkin.catalog;
import jakarta.validation.Valid;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.PositiveOrZero;
import jakarta.validation.constraints.Size;
import java.security.Principal;
import java.util.List;
import java.util.UUID;
import org.springframework.context.annotation.Profile;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PatchMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
```

```
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
@RestController
@Profile("api")
@RequestMapping("/api/v1/booklists")
public class BooklistController {
    private final BooklistService booklists;
    public BooklistController(BooklistService booklists) {
        this.booklists = booklists;
    }
    @GetMapping
    BooklistRepository.BooklistList list(@RequestParam(required = false) String q, Principal principal) {
        return booklists.list(q, principal);
    }
    @PostMapping
    BooklistRepository.BooklistDetail create(@Valid @RequestBody CreateBooklistRequest input, Principal p
rincipal) {
        return booklists.create(input.kind(), input.visibility(), input.title(), input.description(), pri
ncipal);
    }
    @GetMapping("/{id}")
    BooklistRepository.BooklistDetail get(@PathVariable UUID id, Principal principal) {
        return booklists.get(id, principal);
    }
    @PatchMapping("/{id}")
    BooklistRepository.BooklistDetail update(@PathVariable UUID id,
                                             @Valid @RequestBody UpdateBooklistRequest input,
                                             Principal principal) {
        return booklists.update(id, input.title(), input.description(), input.visibility(), input.revisio
n(), principal);
    }
    @DeleteMapping("/{id}")
    ResponseEntity<Void> delete(@PathVariable UUID id, @RequestParam @PositiveOrZero long revision,
                                Principal principal) {
        booklists.delete(id, revision, principal);
        return ResponseEntity.noContent().build();
    }
    @GetMapping("/{id}/books")
    BrowseBookPage books(@PathVariable UUID id,
                         @RequestParam(required = false) String q,
                         @RequestParam(required = false) BookFormat format,
                         @RequestParam(required = false) String cursor,
                         @RequestParam(defaultValue = "36") int limit,
                         Principal principal) {
        return booklists.books(id, q, format, cursor, limit, principal);
    }
    @PostMapping("/{id}/books")
    BooklistRepository.BooklistDetail addBooks(@PathVariable UUID id,
                                               @Valid @RequestBody AddBooklistBooksRequest input,
                                               Principal principal) {
        return booklists.addBooks(id, input.bookIds(), input.revision(), principal);
```

```
    }
    @DeleteMapping("/{id}/books/{bookId}")
    BooklistRepository.BooklistDetail removeBook(@PathVariable UUID id, @PathVariable UUID bookId,
                                                 @RequestParam @PositiveOrZero long revision,
                                                 Principal principal) {
        return booklists.removeBook(id, bookId, revision, principal);
    }
    @PutMapping("/{id}/order")
    BooklistRepository.BooklistDetail reorder(@PathVariable UUID id,
                                              @Valid @RequestBody ReorderBooklistRequest input,
                                              Principal principal) {
        return booklists.reorder(id, input.bookIds(), input.revision(), principal);
    }
    public record CreateBooklistRequest(@NotBlank @Size(max = 240) String title,
                                        @Size(max = 4000) String description,
                                        @NotNull BooklistKind kind,
                                        @NotNull BooklistVisibility visibility) {
    }
    public record UpdateBooklistRequest(@NotBlank @Size(max = 240) String title,
                                        @Size(max = 4000) String description,
                                        @NotNull BooklistVisibility visibility,
                                        @PositiveOrZero long revision) {
    }
    public record AddBooklistBooksRequest(@NotNull List<@NotNull UUID> bookIds,
                                          @PositiveOrZero long revision) {
    }
    public record ReorderBooklistRequest(@NotNull List<@NotNull UUID> bookIds,
                                         @PositiveOrZero long revision) {
    }
}
package com.elexvx.bookkin.catalog;
import jakarta.validation.Valid;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Size;
import java.security.Principal;
import java.util.List;
import java.util.UUID;
import org.springframework.context.annotation.Profile;
import org.springframework.http.ResponseEntity;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PatchMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
```

```
import org.springframework.web.bind.annotation.RestController;
@RestController
@Profile("api")
@RequestMapping("/api/v1")
public class CategoryController {
    private final CategoryService categories;
    public CategoryController(CategoryService categories) {
        this.categories = categories;
    }
    @GetMapping("/categories")
    CategoryRepository.CategoryList list(@RequestParam(required = false) String q, Principal principal) {
        return categories.list(q, principal);
    }
    @PostMapping("/categories")
    @PreAuthorize("hasAnyRole('OWNER','ADMIN')")
    CategoryRepository.CategoryDetail create(@Valid @RequestBody CategoryRequest input, Principal princip
al) {
        return categories.create(input.name(), input.description(), principal);
    }
    @GetMapping("/categories/{id}")
    CategoryRepository.CategoryDetail get(@PathVariable UUID id, Principal principal) {
        return categories.get(id, principal);
    }
    @PatchMapping("/categories/{id}")
    @PreAuthorize("hasAnyRole('OWNER','ADMIN')")
    CategoryRepository.CategoryDetail update(@PathVariable UUID id, @Valid @RequestBody CategoryRequest i
nput,
                                             Principal principal) {
        return categories.update(id, input.name(), input.description(), principal);
    }
    @DeleteMapping("/categories/{id}")
    @PreAuthorize("hasAnyRole('OWNER','ADMIN')")
    ResponseEntity<Void> delete(@PathVariable UUID id, Principal principal) {
        categories.delete(id, principal);
        return ResponseEntity.noContent().build();
    }
    @PutMapping("/categories/order")
    @PreAuthorize("hasAnyRole('OWNER','ADMIN')")
    CategoryRepository.CategoryList reorder(@Valid @RequestBody CategoryOrderRequest input, Principal pri
ncipal) {
        return categories.reorder(input.categoryIds(), principal);
    }
    @GetMapping("/categories/{id}/books")
    BrowseBookPage books(@PathVariable UUID id,
                         @RequestParam(required = false) String q,
                         @RequestParam(required = false) BookFormat format,
                         @RequestParam(defaultValue = "RECENT") BookSort sort,
                         @RequestParam(required = false) String cursor,
                         @RequestParam(defaultValue = "36") int limit,
                         Principal principal) {
        return categories.books(id, q, format, sort, cursor, limit, principal);
    }
    @GetMapping("/books/{bookId}/categories")
```

```
    CategoryRepository.CategoryList categoriesForBook(@PathVariable UUID bookId, Principal principal) {
        return categories.categoriesForBook(bookId, principal);
    }
    @PutMapping("/books/{bookId}/categories")
    @PreAuthorize("hasAnyRole('OWNER','ADMIN')")
    CategoryRepository.CategoryList replaceBookCategories(@PathVariable UUID bookId,
                                                           @Valid @RequestBody BookCategoriesRequest input
,
                                                           Principal principal) {
        return categories.replaceBookCategories(bookId, input.categoryIds(), principal);
    }
    public record CategoryRequest(@NotBlank @Size(max = 160) String name,
                                  @Size(max = 2000) String description) {
    }
    public record CategoryOrderRequest(@NotNull List<@NotNull UUID> categoryIds) {
    }
    public record BookCategoriesRequest(@NotNull List<@NotNull UUID> categoryIds) {
    }
}
package com.elexvx.bookkin.catalog;
import com.elexvx.bookkin.common.ApiException;
import java.nio.file.Files;
import java.nio.file.LinkOption;
import java.nio.file.Path;
import java.security.Principal;
import java.time.Duration;
import java.util.List;
import java.util.Locale;
import java.util.UUID;
import jakarta.validation.Valid;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.PositiveOrZero;
import org.springframework.context.annotation.Profile;
import org.springframework.core.io.FileSystemResource;
import org.springframework.http.CacheControl;
import org.springframework.http.MediaType;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
@RestController
@Profile("api")
@RequestMapping("/api/v1/display-books")
public class DisplayCatalogController {
    private final DisplayCatalogService display;
```

```
    private final BookRepository books;
    public DisplayCatalogController(DisplayCatalogService display, BookRepository books) {
        this.display = display;
        this.books = books;
    }
    @GetMapping
    DisplayCatalogRepository.DisplayBookPage list(@RequestParam(required = false) String q,
                                                  @RequestParam(required = false) String cursor,
                                                  @RequestParam(defaultValue = "60") int limit) {
        return display.list(q, cursor, limit);
    }
    @GetMapping("/{id}")
    DisplayCatalogRepository.DisplayBook get(@PathVariable UUID id) {
        return display.findPublic(id);
    }
    @GetMapping("/{id}/cover")
    ResponseEntity<FileSystemResource> cover(@PathVariable UUID id) {
        // Looking up the public relationship first is intentional: a private book's
        // cached cover must not become an anonymous enumeration endpoint.
        display.findPublic(id);
        var location = books.findCover(id).orElseThrow(() -> ApiException.notFound("COVER_NOT_FOUND", "尚
未生成封面。"));
        Path root;
        Path cover;
        try {
            root = Path.of(location.canonicalRoot() == null ? location.configuredRoot() : location.canoni
calRoot()).toRealPath();
            Path candidate = root.resolve(location.relativePath()).normalize();
            if (!candidate.startsWith(root) || Files.isSymbolicLink(candidate)
                    || !Files.isRegularFile(candidate, LinkOption.NOFOLLOW_LINKS)) {
                throw ApiException.notFound("COVER_NOT_FOUND", "封面缓存不存在。");
            }
            cover = candidate.toRealPath();
            if (!cover.startsWith(root)) throw ApiException.notFound("COVER_NOT_FOUND", "封面缓存不存在。
");
        } catch (java.io.IOException | RuntimeException exception) {
            if (exception instanceof ApiException apiException) throw apiException;
            throw ApiException.notFound("COVER_NOT_FOUND", "封面缓存不存在。");
        }
        String lower = cover.getFileName().toString().toLowerCase(Locale.ROOT);
        MediaType type = lower.endsWith(".png") ? MediaType.IMAGE_PNG
                : lower.endsWith(".webp") ? MediaType.parseMediaType("image/webp") : MediaType.IMAGE_JPEG
;
        return ResponseEntity.ok().contentType(type)
                .cacheControl(CacheControl.maxAge(Duration.ofDays(30)).cachePublic())
                .body(new FileSystemResource(cover));
    }
    @PostMapping
    DisplayCatalogRepository.DisplayBook add(@Valid @RequestBody AddRequest input, Principal principal) {
        return display.add(input.bookId(), input.revision(), principal);
    }
    @DeleteMapping("/{id}")
    ResponseEntity<Void> remove(@PathVariable UUID id, @RequestParam @PositiveOrZero long revision, Princ
ipal principal) {
        display.remove(id, revision, principal);
```

```
        return ResponseEntity.noContent().build();
    }
    @PutMapping("/order")
    DisplayCatalogRepository.DisplayBookPage reorder(@Valid @RequestBody OrderRequest input, Principal pr
incipal) {
        return display.reorder(input.bookIds(), input.revision(), principal);
    }
    public record AddRequest(@NotNull UUID bookId, @PositiveOrZero long revision) {}
    public record OrderRequest(@PositiveOrZero long revision, @NotNull List<@NotNull UUID> bookIds) {}
}
package com.elexvx.bookkin.catalog;
import com.elexvx.bookkin.catalog.BookMetadataRepository.MetadataPatch;
import com.elexvx.bookkin.common.ApiException;
import com.elexvx.bookkin.filemanagement.FileOperationModels.Preview;
import com.elexvx.bookkin.filemanagement.FileOperationModels.PreviewRequest;
import com.elexvx.bookkin.filemanagement.FileOperationService;
import com.elexvx.bookkin.filemanagement.FileOperationType;
import jakarta.validation.Valid;
import jakarta.validation.constraints.Size;
import java.security.Principal;
import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.List;
import java.util.UUID;
import org.springframework.context.annotation.Profile;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.web.bind.annotation.PatchMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.http.MediaType;
@RestController
@Profile("api")
@RequestMapping("/api/v1/books")
@PreAuthorize("hasAnyRole('OWNER','ADMIN')")
public class MetadataController {
    private final BookMetadataRepository metadata;
    private final BookRepository books;
    private final FileOperationService fileOperations;
    private final BookCoverService covers;
    public MetadataController(BookMetadataRepository metadata, BookRepository books, FileOperationService
 fileOperations, BookCoverService covers) {
        this.metadata = metadata;
        this.books = books;
        this.fileOperations = fileOperations;
        this.covers = covers;
    }
    @GetMapping("/{id}/metadata")
```